Arrays - Pjesa 2

Programim në gjuhën C

Alion Emini

kurseinformatike.com

Përmbajtja

1. Hyrje
2. Si ruhen matricat në memorie
3. Ciklet `for` të mbivendosura
4. Inicializimi i matricave
5. Funksionet me matricat
6. Array-t konstante (`const`)
7. Variable-Length Arrays (C99)
8. Ushtrime
9. Përmbledhje

Përmbajtja

1. Hyrje
2. Si ruhen matricat në memorie
3. Ciklet `for` të mbivendosura
4. Inicializimi i matricave
5. Funksionet me matricat
6. Array-t konstante (`const`)
7. Variable-Length Arrays (C99)

Çfarë janë matricat?

Ideja

Një array mund të ketë **çdo numër dimensioni**. Kur kemi **dy dimensione**, e quajmë **matricë** (në terminologjinë matematikore).

Sintaksa dhe deklarimi shembull

```
type name[ROWS][CLS];
int m[4][3];
```

`m` ka **4 rreshta** dhe **3 kolona**. Indekset fillojnë nga **0**.

Indekset në një matricë `m[4][3]` I

Si mendohet?

- Matrica është një **tabelë** me **rreshta** dhe **kolona**.
- Çdo qelizë identifikohet me dy numra: `(rreshti, kolona)`.
- Në C, indekset fillojnë nga **0**.

Shënim

Për `m[4][3]` kemi rreshtat `0..3` dhe kolonat `0..2`.

Indekset në një matricë `m[4][3]` II

[0][0]	[0][1]	[0][2]
[1][0]	[1][1]	[1][2]
[2][0]	[2][1]	[2][2]
[3][0]	[3][1]	[3][2]

Vetëm indekset

1 [0][0]	2 [0][1]	3 [0][2]
4 [1][0]	5 [1][1]	6 [1][2]
7 [2][0]	8 [2][1]	9 [2][2]
10 [3][0]	11 [3][1]	12 [3][2]

Vlerat + indekset

Si lexohet `m[i][j]`?

Leximi

`m[i][j]` do të thotë:

- shko te **rreshti** me indeks `i`
- pastaj te **kolona** me indeks `j`

Mbaje mend

`i` është gjithmonë rreshti, `j` është gjithmonë kolona.

Shembull vizual: `m[2][1]`

Çfarë qelie është?

`m[2][1]` = rreshti `i=2` (rreshti i tretë) dhe kolona `j=1` (kolona e dytë).

[0][0]	[0][1]	[0][2]
[1][0]	[1][1]	[1][2]
[2][0]	[2][1]	[2][2]
[3][0]	[3][1]	[3][2]

Praktikë

1. Nëse do të lexosh këtë qelizë: `x = m[2][1];`
2. Nëse do të shkruash: `m[2][1] = 10;`

Shembull vizual: `m[1][0]`

Çfarë qelie është?

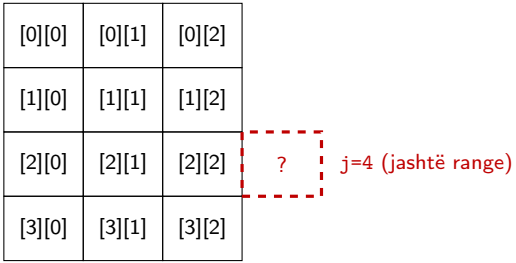
`m[1][0]` = rreshti `i=1` (rreshti i dytë) dhe kolona `j=0` (kolona e parl).

[0][0]	[0][1]	[0][2]
[1][0]	[1][1]	[1][2]
[2][0]	[2][1]	[2][2]
[3][0]	[3][1]	[3][2]

Shembull vizual: `m[2][4]` (indeks që nuk ekziston)

Kujto dimensionet

Për `m[4][3]`: rreshtat: `0..3`, kolonat: `0..2`



Gabim: out-of-bounds

`m[2][4]` nuk ekziston, sepse kolona 4 është jashtë `0..2`. Në C kjo sjell **undefined behavior**.

m[i][j] dhe jo m[i, j] I

Aksesimi i saktë

```
int x = m[i][j];
m[i][j] = 10;
```

Çfarë do të thotë m[i]?

m[i] përfaqëson **rreshtin** i. Pastaj m[i][j] zgjedh elementin j të atij rreshti.

m[i][j] dhe jo m[i, j] II

Gabim i zakonshëm: presja

```
/* GABIM: presja është operator (comma operator) */
x = m[i, j]; /* E njëjta si: x = m[j]; */
```

Pse?

Në C, `(i, j)` vlerëson `i` dhe pastaj kthen `j`. Prandaj `m[i, j]` bëhet `m[j]`.

Përmbajtja

1. Hyrje
2. Si ruhen matricat në memorie
3. Ciklet `for` të mbivendosura
4. Inicializimi i matricave
5. Funksionet me matricat
6. Array-t konstante (`const`)
7. Variable-Length Arrays (C99)

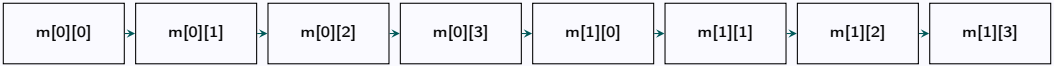
Row-major order (renditja sipas rreshtave)

Fakt i rëndësishëm

Edhe pse ne e vizualizojmë matricën si tabelë, në memorie elementet ruhen **njëpasnjëshëm** në **row-major order**:

$$m[0][0], m[0][1], \ldots, m[0][C-1], m[1][0], \ldots$$

Shembull për `m[2][4]`



Pse ka rëndësi row-major?

Performancë (cache locality)

Kur përpunon matricën, është zakonisht më efikase të ecësh:

- **rresht pas rreshti**: `row` jashtë, `col` brenda.
- sepse elementet e një rreshti janë pranë njëri-tjetrit në memorie.

Modeli tipik

```
for (row = 0; row < ROWS; row++)
    for (col = 0; col < COLS; col++) ...
```

Përmbajtja

1. Hyrje
2. Si ruhen matricat në memorie
3. Ciklet `for` të mbivendosura
4. Inicializimi i matricave
5. Funksionet me matricat
6. Array-t konstante (`const`)
7. Variable-Length Arrays (C99)

Përpunimi i matricave me `for` të dyfishtë

Ideja

Ashtu si `for` është ideal për array 1D, **for-i i mbivendosur** është ideal për array 2D:

- cikli i jashtëm: rreshti
- cikli i brendshëm: kolona

Skemë

- Vizitojmë çdo qelizë `m[row][col]` në mënyrë sistematike.
- Mund të inicializojmë, llogarisim, kërkojmë, numërojmë, etj.

Shembulli 1: Matricë identiteti N×N I

Qëllimi

1.0 në diagonalën kryesore (`row == col`), **0.0** kudo tjetër.

```
#include <stdio.h>
#define N 8
int main(void) {
    double ident[N][N];
    int row, col;

    /* Inicializimi */
    for (row = 0; row < N; row++) {
        for (col = 0; col < N; col++) {
            if (row == col) ident[row][col] = 1.0;
            else ident[row][col] = 0.0;
        }
    }
```

Shembulli 1: Matricë identiteti N×N II

```
    /* Printimi */
    for (row = 0; row < N; row++) {
        for (col = 0; col < N; col++)
            printf("%.1f ", ident[row][col]);
        printf("\n");
    }

    return 0;
}
```

Shembulli 1: Matricë identiteti N×N III

Vizualizim (N=8)

1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	1

Shembulli 1: Iterimi 1

1							

Shpjegimi

```
row = 0, col = 0
ident[0][0] = 1
```

Shembulli 1: Iterimi 2

1	0						

Shpjegimi

```
row = 0, col = 1
ident[0][1] = 0
```

Shembulli 1: Iterimi 3

1	0	0					

Shpjegimi

```
row = 0, col = 2
ident[0][2] = 0
```

Shembulli 1: Iterimi 4

1	0	0	0				

Shpjegimi

```
row = 0, col = 3
ident[0][3] = 0
```

Shembulli 1: Iterimi 5

1	0	0	0	0			

Shpjegimi

```
row = 0, col = 4
ident[0][4] = 0
```

Shembulli 1: Iterimi 6

1	0	0	0	0	0		

Shpjegimi

```
row = 0, col = 5
ident[0][5] = 0
```

Shembulli 1: Iterimi 7

1	0	0	0	0	0	0	

Shpjegimi

```
row = 0, col = 6
ident[0][6] = 0
```

Shembulli 1: Iterimi 8

1	0	0	0	0	0	0	0

Shpjegimi

```
row = 0, col = 7
ident[0][7] = 0
```

Shembulli 1: Iterimi 9

1	0	0	0	0	0	0	0
0							

Shpjegimi

```
row = 1, col = 0
ident[1][0] = 0
```

Shembulli 1: Iterimi 10

1	0	0	0	0	0	0	0
0	1						

Shpjegimi

```
row = 1, col = 1
ident[1][1] = 1
```

Shembulli 1: Iterimi 11

1	0	0	0	0	0	0	0
0	1	0					

Shpjegimi

```
row = 1, col = 2
ident[1][2] = 0
```

Shembulli 1: Iterimi 12

1	0	0	0	0	0	0	0
0	1	0	0				

Shpjegimi

```
row = 1, col = 3
ident[1][3] = 0
```

Shembulli 1: Iterimi 13

1	0	0	0	0	0	0	0
0	1	0	0	0			

Shpjegimi

```
row = 1, col = 4
ident[1][4] = 0
```

Shembulli 1: Iterimi 14

1	0	0	0	0	0	0	0
0	1	0	0	0	0		

Shpjegimi

```
row = 1, col = 5
ident[1][5] = 0
```

Shembulli 1: Iterimi 15

1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	

Shpjegimi

```
row = 1, col = 6
ident[1][6] = 0
```

Shembulli 1: Iterimi 16

1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0

Shpjegimi

```
row = 1, col = 7
ident[1][7] = 0
```

Shembulli 1: Iterimi 17

1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0
0							

Shpjegimi

```
row = 2, col = 0
ident[2][0] = 0
```

Shembulli 1: Iterimi 18

1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0
0	0						

Shpjegimi

```
row = 2, col = 1
ident[2][1] = 0
```

Shembulli 1: Iterimi 19

1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0
0	0	1					

Shpjegimi

```
row = 2, col = 2
ident[2][2] = 1
```

Shembulli 1: Iterimi 20

1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0
0	0	1	0				

Shpjegimi

```
row = 2, col = 3
ident[2][3] = 0
```

Shembulli 2: Shuma e të gjithë elementeve

```c
#include <stdio.h>
#define ROWS 3
#define COLS 3

int main(void) {
    int matrix[ROWS][COLS] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };
    int sum = 0;

    for (int i = 0; i < ROWS; i++)
        for (int j = 0; j < COLS; j++)
            sum += matrix[i][j];

    printf("Shuma: %d\n", sum);
    return 0;
}
```

Shembulli 2: Iterimi 1/9

Shpjegimi

```
i = 0, j = 0
matrix[0][0] = 1
sum:  0 + 1 = 1
```

1		

Shembulli 2: Iterimi 2/9

Shpjegimi

```
i = 0, j = 1
matrix[0][1] = 2
sum:  1 + 2 = 3
```

1	2	

Shembulli 2: Iterimi 3/9

Shpjegimi

```
i = 0, j = 2
matrix[0][2] = 3
sum:  3 + 3 = 6
```

1	2	3

Shembulli 2: Iterimi 4/9

Shpjegimi

```
i = 1, j = 0
matrix[1][0] = 4
sum:  6 + 4 = 10
```

1	2	3
4		

Shembulli 2: Iterimi 5/9

Shpjegimi

```
i = 1, j = 1
matrix[1][1] = 5
sum:  10 + 5 = 15
```

1	2	3
4	5	

Shembulli 2: Iterimi 6/9

Shpjegimi

```
i = 1, j = 2
matrix[1][2] = 6
sum:  15 + 6 = 21
```

1	2	3
4	5	6

Shembulli 2: Iterimi 7/9

Shpjegimi

```
i = 2, j = 0
matrix[2][0] = 7
sum:  21 + 7 = 28
```

1	2	3
4	5	6
7		

Shembulli 2: Iterimi 8/9

Shpjegimi

```
i = 2, j = 1
matrix[2][1] = 8
sum:  28 + 8 = 36
```

1	2	3
4	5	6
7	8	

Shembulli 2: Iterimi 9/9

Shpjegimi

```
i = 2, j = 2
matrix[2][2] = 9
sum:  36 + 9 = 45
```

1	2	3
4	5	6
7	8	9

Përmbajtja

1. Hyrje
2. Si ruhen matricat në memorie
3. Ciklet `for` të mbivendosura
4. Inicializimi i matricave
5. Funksionet me matricat
6. Array-t konstante (`const`)
7. Variable-Length Arrays (C99)

Inicializimi me inicializues të mbivendosur

Forma standarde

Çdo listë e brendshme përfaqëson një **rresht**.

```
int m[5][9] = {
    {1, 1, 1, 1, 1, 0, 1, 1, 1},
    {0, 1, 0, 1, 0, 1, 0, 1, 0},
    {0, 1, 0, 1, 1, 0, 0, 1, 0},
    {1, 1, 0, 1, 0, 0, 0, 1, 0},
    {1, 1, 0, 1, 0, 0, 1, 1, 1}
};
```

Rregull i përgjithshëm

Nëse mungojnë elementë, pjesa e mbetur inicializohet automatikisht me **0**.

Shkurtimi i inicializuesve: rregullat I

1) Mungojnë rreshta të tërë

Rreshtat e mbetur bëhen **0**.

```
int m[5][9] = {
    {1, 1, 1, 1, 1, 0, 1, 1, 1},
    {0, 1, 0, 1, 0, 1, 0, 1, 0},
    {0, 1, 0, 1, 1, 0, 0, 1, 0}
    /* 2 rreshta të fundit -> 0 */
};
```

Shkurtimi i inicializuesve: rregullat II

1	1	1	1	1	0	1	1	1
0	1	0	1	0	1	0	1	0
0	1	0	1	1	0	0	1	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Shkurtimi i inicializuesve: rregullat III

2) Mungojnë elementë brenda rreshtit

Elementët e mbetur në atë rresht inicializohen me **0**.

```
int m[5][9] = {
    {1, 1, 1, 1, 1, 0, 1, 1, 1},
    {0, 1, 0, 1, 0, 1, 0, 1}, /* mungon elementi i 9-të -> 0 */
    {0, 1, 0, 1, 1, 0, 0, 1, 0},
    {1, 1, 0, 1, 0, 0, 0, 1}, /* pjesa tjetër -> 0 */
    {1, 1, 0, 1, 0, 0, 1, 1, 1}
};
```

Shkurtimi i inicializuesve: rregullat IV

1	1	1	1	1	0	1	1	1
0	1	0	1	0	1	0	1	0
0	1	0	1	1	0	0	1	0
1	1	0	1	0	0	0	1	0
1	1	0	1	0	0	1	1	1

Shkurtimi i inicializuesve: rregullat V

3) Heqja e kllapave të brendshme (lejohet, por e rrezikshme)

```
int m[5][9] = {
    1, 1, 1, 1, 1, 0, 1, 1, 1,
    0, 1, 0, 1, 0, 1, 0, 1, 0,
    0, 1, 0, 1, 1, 0, 0, 1, 0,
    1, 1, 0, 1, 0, 0, 0, 1, 0,
    1, 1, 0, 1, 0, 0, 1, 1, 1
};
```

Shkurtimi i inicializuesve: rregullat VI

1	1	1	1	1	0	1	1	1
0	1	0	1	0	1	0	1	0
0	1	0	1	1	0	0	1	0
1	1	0	1	0	0	0	1	0
1	1	0	1	0	0	1	1	1

Kujdes

Një element ekstra ose mungesë ndryshon gjithë "zhvendosjen" e të dhënave.

Inicializues të përcaktuar (C99): matricë identiteti 2×2 I

Shembull

```c
int ident[2][2] = {
    [0][0] = 1,
    [1][1] = 1
};
/* Të tjerat -> 0.0 */
```

Avantazh

Shkruan vetëm elementët që të duhen; pjesa tjetër shkon në 0 automatikisht.

Inicializues të përcaktuar (C99): matricë identiteti 2×2 II

1	0
0	1

Përmbajtja

1. Hyrje
2. Si ruhen matricat në memorie
3. Ciklet `for` të mbivendosura
4. Inicializimi i matricave
5. Funksionet me matricat
6. Array-t konstante (`const`)
7. Variable-Length Arrays (C99)

Funksionet me matricat (shumë e rëndësishme)

Pse është "pika ku ngecet" më shpesh?

Kur një matricë i kalon një funksioni, ajo **nuk** bart automatikisht dimensionet e saj. Prandaj:

- duhen dimensionet (të paktën **kolonat**) që kompiluesi të llogarisë adresimin,
- dhe shpesh duhet t'i kalojmë **rows/cols** si parametra.

Rregull praktik

Te parametrat e funksionit, një array "**degradohet**" në pointer, prandaj `sizeof` s'na jep madhësinë totale të matricës.

Pse sizeof s'punon në parametra (2D)?

Ideja

Në funksion, parametri `int m[][COLS]` trajtohet si pointer te një rresht: `int (*m)[COLS]`.

```c
#include <stdio.h>

#define ROWS 4
#define COLS 3

void g(int m[ROWS][COLS]) {
    printf("sizeof(m) = %zu\n", sizeof(m));
    printf("sizeof(m[0]) = %zu\n", sizeof(m[0]));
    printf("sizeof(m[0][0])= %zu\n", sizeof(m[0][0]));
}

int main(void) {
    int a[ROWS][COLS];
    printf("sizeof(a) = %zu\n", sizeof(a));
    g(a);
}
```

Si deklarohet parametri i një matrice 2D?

Tre forma tipike

```c
/* 1) Me makro/konstante të njohura në kompilim */
void f(int m[ROWS][COLS]);

/* 2) Kolonat fikse, rreshtat i kalojmë si parametër */
void f(int m[][COLS], int rows);

/* 3) C99 (VLA): rreshtat/kolonat si parametra */
void f(int rows, int cols, int m[rows][cols]);
```

Shënim

Në shumicën e rasteve, **dimensioni i dytë** (`COLS`) duhet të jetë i njohur që të funksionojë `m[i][j]`.

Shembulli 3: print_matrix

```c
#include <stdio.h>

#define COLS 3

void print_matrix(int rows, int m[][COLS]) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < COLS; j++)
            printf("%d ", m[i][j]);
        printf("\n");
    }
}

int main(void) {
    int a[2][COLS] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    print_matrix(2, a);
    return 0;
}
```

Përmbajtja

1. Hyrje
2. Si ruhen matricat në memorie
3. Ciklet `for` të mbivendosura
4. Inicializimi i matricave
5. Funksionet me matricat
6. Array-t konstante (`const`)
7. Variable-Length Arrays (C99)

const me array

Deklarim

```
const char hex_chars[] = {
    '0','1','2','3','4','5','6','7','8','9',
    'A','B','C','D','E','F'
};
```

'0'	'1'	'2'	'3'	'4'	'5'	'6'	'7'	'8'	'9'	'A'	'B'	'C'	'D'	'E'	'F'

Përfaqësimi vizual i array-t `hex_chars` (16 elemente)

Çfarë garanton const?

- Programi **nuk duhet** ta modifikojë array-n.
- Kompilatori kap gabimet kur përpiqesh ta ndryshosh.
- Dokumenton që array është "referencë" (të dhëna fikse).

const edhe në funksione

Model standard: mos e ndrysho array-n

```
int sum(const int a[], int n) {
    int s = 0;
    for (int i = 0; i < n; i++)
        s += a[i];
    return s;
}
```

Pse është e rëndësishme?

E bën kontratën të qartë: funksioni lexon, por nuk shkruan.

Program 1: Ideja

Qëllimi

Shpërndan një dorë kartash të rastësishme nga një kuvertë standarde, pa duplikime.

Çfarë përdorim?

- `bool in_hand[NUM_SUITS][NUM_RANKS]` për të shënuar kartat e shpërndara.
- `const char rank_code[]` dhe `const char suit_code[]` për kodet e kartave.

Program 1: Kodi I

```c
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define NUM_SUITS 4
#define NUM_RANKS 13

int main(void) {
    bool in_hand[NUM_SUITS][NUM_RANKS] = {false};
    int num_cards, rank, suit;

    const char rank_code[] = {'2','3','4','5','6','7','8','9','T','J','Q','K','A'};
    const char suit_code[] = {'c','d','h','s'};

    srand((unsigned) time(NULL));

    printf("Shkruani numrin e kartave ne dore: ");
    scanf("%d", &num_cards);

    printf("Dora juaj:");
```

Program 1: Kodi II

```
    while (num_cards > 0) {
        suit = rand() % NUM_SUITS;
        rank = rand() % NUM_RANKS;

        if (!in_hand[suit][rank]) {
            in_hand[suit][rank] = true;
            num_cards--;
            printf(" %c%c", rank_code[rank], suit_code[suit]);
        }
    }

    printf("\n");
    return 0;
}
```

Pika kyçe

`in_hand[suit][rank]` garanton që e njëjta kartë nuk del dy herë.

Program 1: Inputet e supozuara

Input (nga përdoruesi)

- `num_cards = 3`

Supozim për `rand()` (për ta ndjekur ekzekutimin)

Në çdo iterim të `while (num_cards > 0)` supozojmë që `rand()` jep këto çifte `(suit, rank)`:

1. (2, 8) ⇒ `'h'`, `'T'` ⇒ `Th`
2. (0, 12) ⇒ `'c'`, `'A'` ⇒ `Ac`
3. (2, 8) ⇒ `Th` (duplikim)
4. (3, 0) ⇒ `'s'`, `'2'` ⇒ `2s`

Program 1: Ekzekutimi (Iterimi 1)

Para iterimit

`num_cards = 3` dhe të gjitha `in_hand[suit][rank] = false`.

Gjenerimi

`suit = 2`, `rank = 8` $\Rightarrow$ karta `Th`.

Kontrolli

`!in_hand[2][8]` është **true** $\Rightarrow$ prano kartën.

Veprimi

```
in_hand[2][8] = true;
num_cards- ⇒ num_cards = 2
Printo: Th
```

Program 1: Ekzekutimi (Iterimi 2)

Para iterimit

`num_cards = 2` dhe në dorë kemi: `Th`.

Gjenerimi

`suit = 0`, `rank = 12` $\Rightarrow$ karta `Ac`.

Kontrolli

`!in_hand[0][12]` është **true** $\Rightarrow$ prano kartën.

Veprimi

```
in_hand[0][12] = true;
num_cards- ⇒ num_cards = 1
Printo: Th Ac
```

Program 1: Ekzekutimi (Iterimi 3 — duplikim)

Para iterimit

`num_cards = 1` dhe në dorë kemi: `Th Ac`.

Gjenerimi

`suit = 2`, `rank = 8` $\Rightarrow$ karta `Th` (sërish).

Kontrolli

`!in_hand[2][8]` është **false** sepse kjo kartë është marrë më herët.

Veprimi

Nuk hyjmë në `if` $\Rightarrow$:
`num_cards` mbetet 1 dhe nuk printohet asgjë.

Program 1: Ekzekutimi (Iterimi 4)

Para iterimit

`num_cards = 1` dhe në dorë kemi: `Th Ac`.

Gjenerimi

`suit = 3`, `rank = 0` $\Rightarrow$ karta 2s.

Kontrolli

`!in_hand[3][0]` është **true** $\Rightarrow$ prano kartën.

Veprimi + Përfundimi

`in_hand[3][0] = true;`

`num_cards-` $\Rightarrow$ `num_cards = 0` $\Rightarrow$ cikli ndalet.

Output final: `Th Ac 2s`

Përmbajtja

1. Hyrje
2. Si ruhen matricat në memorie
3. Ciklet `for` të mbivendosura
4. Inicializimi i matricave
5. Funksionet me matricat
6. Array-t konstante (`const`)
7. Variable-Length Arrays (C99)

VLAs (C99)

Çfarë janë VLAs?

Array me gjatësi të ndryshueshme: gjatësia përcaktohet **në kohë ekzekutimi** (runtime), jo në kompilim.

Përparësi

- Fleksibilitet: përdor madhësinë reale që të duhet.
- Memorie: shmang madhësi arbitrare.

Kufizime tipike

- Nuk kanë kohëzgjatje statike.
- Nuk mund të kenë inicializues në deklarim.
- Në C11 e më tej, mbështetja për VLA mund të jetë **opsionale**.

Shembulli 4 me VLA I

```c
#include <stdio.h>

int main(void) {
    int n, i;

    printf("Sa numra deshironi te ktheni? ");
    scanf("%d", &n);

    int a[n]; /* C99 only - gjatesia varet nga n */

    printf("Shkruani %d numra: ", n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);

    printf("Ne rend te kundert:");
    for (i = n - 1; i >= 0; i--) printf(" %d", a[i]);
    printf("\n");

    return 0;
}
```

Shembulli 4 me VLA II

Shënim praktik

VLA ruhet zakonisht në stack: nëse `n` është shumë i madh, rrezikon stack overflow.

VLA shumëdimensionale I

Edhe matricat mund të jenë VLA

```c
int m, n;
scanf("%d%d", &m, &n);

int a[m][n]; /* VLA 2D */
```

1 [0][0]	1 [0][1]	1 [0][2]	1 [0][3]
0 [1][0]	1 [1][1]	0 [1][2]	1 [1][3]
0 [2][0]	1 [2][1]	0 [2][2]	1 [2][3]

VLA shumëdimensionale II

Kujdes

Për funksione, dimensioni i dytë zakonisht duhet të njihet (ose të kalojë si parametër) për adresimin korrekt.

Përmbajtja

1. Hyrje
2. Si ruhen matricat në memorie
3. Ciklet `for` të mbivendosura
4. Inicializimi i matricave
5. Funksionet me matricat
6. Array-t konstante (`const`)
7. Variable-Length Arrays (C99)

Ushtrimi 1: Totalet e rreshtave dhe kolonave

Detyra

Lexo një matricë **5×5** me numra të plotë dhe printo:

- totalet e çdo rreshti,
- totalet e çdo kolone.

Shembull output

```
Row totals:  30 27 40 36 28
Column totals:  34 37 37 32 21
```

Ushtrimi 1: Zgjidhja I

```c
#include <stdio.h>

int main(void) {
    int m[5][5];

    for (int i = 0; i < 5; i++) {
        printf("Enter row %d: ", i + 1);
        for (int j = 0; j < 5; j++)
            scanf("%d", &m[i][j]);
    }

    printf("Row totals: ");
    for (int i = 0; i < 5; i++) {
        int sum = 0;
        for (int j = 0; j < 5; j++)
            sum += m[i][j];
        printf("%d ", sum);
    }
```

Ushtrimi 1: Zgjidhja II

```
    printf("\nColumn totals: ");
    for (int j = 0; j < 5; j++) {
        int sum = 0;
        for (int i = 0; i < 5; i++)
            sum += m[i][j];
        printf("%d ", sum);
    }

    printf("\n");
    return 0;
}
```

Ushtrimi 2: Nota kuizi (studentë $\times$ kuize)

Detyra

Lexo **5 nota kuizi** për secilin nga **5 studentët**. Pastaj:

- llogarit totalin dhe mesataren për secilin student,
- llogarit mesataren, notën më të lartë dhe më të ulët për secilin kuiz.

Ushtrimi 2: Zgjidhja I

```c
#include <stdio.h>

int main(void) {
    int grades[5][5];

    for (int s = 0; s < 5; s++) {
        printf("Enter grades for student %d: ", s + 1);
        for (int q = 0; q < 5; q++)
            scanf("%d", &grades[s][q]);
    }

    printf("\nStudent totals and averages:\n");
    for (int s = 0; s < 5; s++) {
        int sum = 0;
        for (int q = 0; q < 5; q++)
            sum += grades[s][q];
        printf("Student %d: Total = %d, Average = %.2f\n",
               s + 1, sum, sum / 5.0);
    }
```

Ushtrimi 2: Zgjidhja II

```
    printf("\nQuiz statistics:\n");
    for (int q = 0; q < 5; q++) {
        int sum = 0;
        int min = grades[0][q], max = grades[0][q];

        for (int s = 0; s < 5; s++) {
            int val = grades[s][q];
            sum += val;
            if (val < min) min = val;
            if (val > max) max = val;
        }

        printf("Quiz %d: Average = %.2f, High = %d, Low = %d\n",
               q + 1, sum / 5.0, max, min);
    }

    return 0;
}
```

Ushtrimi 3: Random Walk (10×10)

Detyra

Gjenero një "random walk" në një matricë **10×10**:

- inicializo me `'.'`,
- vendos `'A'` në `(0,0)`,
- për `'B'` deri `'Z'` lëviz rastësisht (lart/djathtas/poshtë/majtas),
- mos dil jashtë kufijve dhe mos shkel qeliza të mbushura,
- nëse bllokohesh në të 4 drejtimet, ndalo.

Ushtrimi 3: Zgjidhja I

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    char grid[10][10];

    for (int i = 0; i < 10; i++)
        for (int j = 0; j < 10; j++)
            grid[i][j] = '.';

    int x = 0, y = 0;
    char letter = 'A';
    grid[x][y] = letter;

    srand((unsigned) time(NULL));
```

Ushtrimi 3: Zgjidhja II

```
    while (letter < 'Z') {
        int dirs[4] = {0, 1, 2, 3}; /* 0=lart,1=djathtas,2=poshte,3=majtas */

        for (int i = 3; i > 0; i--) {
            int j = rand() % (i + 1);
            int tmp = dirs[i]; dirs[i] = dirs[j]; dirs[j] = tmp;
        }


        int moved = 0;
```

Ushtrimi 3: Zgjidhja III

```java
        for (int k = 0; k < 4; k++) {
            int nx = x, ny = y;

            switch (dirs[k]) {
                case 0: nx = x - 1; break;
                case 1: ny = y + 1; break;
                case 2: nx = x + 1; break;
                case 3: ny = y - 1; break;
            }

            if (nx >= 0 && nx < 10 && ny >= 0 && ny < 10 && grid[nx][ny] == '.') {
                letter++;
                grid[nx][ny] = letter;
                x = nx; y = ny;
                moved = 1;
                break;
            }
        }
```

Ushtrimi 3: Zgjidhja IV

```
        if (!moved)
            break;
    }

    for (int i = 0; i < 10; i++) {
        for (int j = 0; j < 10; j++)
            printf("%c ", grid[i][j]);
        printf("\n");
    }

    return 0;
}
```

Sugjerim

Për "random walk", përzierja e drejtimeve (shuffle) ul shanset të ngecësh herët dhe e bën ecjen më "të rastësishme".

Përmbajtja

1. Hyrje
2. Si ruhen matricat në memorie
3. Ciklet `for` të mbivendosura
4. Inicializimi i matricave
5. Funksionet me matricat
6. Array-t konstante (`const`)
7. Variable-Length Arrays (C99)

Përmbledhje

Çfarë mësuam?

- Deklarimi dhe indeksimi i matricave: `m[row][col]`.
- Gabimi `m[i, j]` dhe pse është i gabuar.
- Row-major order dhe pse cikli "rresht jashtë / kolonë brenda" është praktik.
- Përpunimi me `for` të mbivendosura: identiteti, shuma, statistika.
- Inicializimi i matricave (të plotë, i pjesshëm, C99 designated).
- `const` arrays dhe VLAs (C99).

Këshillë

Mbaj dimensionet si makro (`#define ROWS ...`) dhe ruaj rendin e cikleve për të shmangur gabime dhe për të pasur kod më të lexueshëm.

Faleminderit!

Pyetje?